Antigravity/Sci-Fi MERN Rental Platform - Master Planner

1. Futuristic UI/UX Design System

Before building, establish the visual identity to ensure the futuristic theme is consistent.

Color Palette:

- **Background (Space / Dark Matte):** `#0B0C10` (Deep Void) or `#121212` (Matte Black)
- **Primary Accent (Neon Cyan):** `#00F0FF` (Hologram Blue) - *Used for primary buttons, active states, and glowing text.*
- **Secondary Accent (Deep Purple):** `#7A04EB` (Nebula Purple) - *Used for secondary buttons, card borders, and gradient blends.*
- **Tertiary/Warning (Laser Red/Orange):** `#FF003C` (Overheat Red) - *Used for errors, low battery warnings, or destructive actions.*
- **Text (Starlight White):** `#E0E6ED` (Crisp, readable off-white).

Typography:

- **Headings:** `Orbitron` or `Rajdhani` (Google Fonts) for that angular, high-tech HUD feel.
- **Body:** `Inter` or `Roboto Mono` for clean, readable technical data.

---

2. Project Structure (Modular & Clean)

Use this exact folder structure to separate concerns (MVC pattern) demonstrating professional best practices.

```
antigravity-rental/
├── backend/
│   ├── config/          # Database connection, environment vars
│   ├── controllers/     # Core business logic (e.g., leaseController.js)
│   ├── middlewares/     # Auth, error handling, gravity clearance checks
│   ├── models/          # Mongoose Schemas (User, Gear, Lease)
│   ├── routes/          # Express route definitions
│   ├── utils/           # Helper functions (date lap checkers)
│   └── server.js        # Entry point
├── frontend/            # React (Vite) + Tailwind Project
│   ├── src/
│   │   ├── assets/      # Images, 3D models, icons
│   │   ├── components/  # Reusable UI (FloatingCard, Navbar, HUD)
│   │   ├── context/     # Global state (AuthContext, CartContext)
│   │   ├── pages/       # Route components (Marketplace, Dashboard)
│   │   ├── services/    # Axios API calls to backend
│   │   ├── App.jsx
│   │   └── index.css    # Global styles, custom Tailwind animations
│   └── tailwind.config.js
└── .env                 # Environment variables
```

---

3. Sci-Fi Database Models (Mongoose)

`User` Model

```
// backend/models/User.js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  email: { type: String, required: true, unique: true },
  passwordHash: { type: String, required: true },

  // Sci-Fi Attributes
  gravityClearance: {
    type: String,
    enum: ['CIVILIAN', 'PILOT', 'COMMANDER', 'ADMIN'],
    default: 'CIVILIAN'
  },
  creditsBalance: { type: Number, default: 1000 }, // Futuristic currency
  flightHours: { type: Number, default: 0 } // Tracks their experience
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

`AntiGravityGear` Model

```
// backend/models/AntiGravityGear.js
const mongoose = require('mongoose');

const gearSchema = new mongoose.Schema({
  name: { type: String, required: true }, // e.g., "Zephyr Hoverboard MK-II"
  type: {
    type: String,
    enum: ['HOVERBOARD', 'GRAVITY_SUIT', 'JETPACK', 'REPULSOR_BOOTS'],
    required: true
  },
  description: { type: String },
  pricePerCycle: { type: Number, required: true }, // 'Cycle' instead of 'Day'
  isAvailable: { type: Boolean, default: true },

  // Sci-Fi Attributes
  thrustPower: { type: Number, required: true }, // e.g., in Newtons or 'kN'
  batteryLife: { type: Number, required: true }, // e.g., in standard hours
  requiredClearance: {
    type: String,
    enum: ['CIVILIAN', 'PILOT', 'COMMANDER'],
    default: 'CIVILIAN'
  },
  techSpecs: {
```

```
    maxSpeed: { type: String }, // e.g., "Mach 1.2"
    weightCapacity: { type: String }
  },
  imageUrl: { type: String }
}, { timestamps: true });

module.exports = mongoose.model('AntiGravityGear', gearSchema);
```

`Lease` Model

```
// backend/models/Lease.js
const mongoose = require('mongoose');

const leaseSchema = new mongoose.Schema({
  gear: { type: mongoose.Schema.Types.ObjectId, ref: 'AntiGravityGear', required: true },
  renter: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },

  // Logistics
  startCycle: { type: Date, required: true },
  endCycle: { type: Date, required: true },
  totalCost: { type: Number, required: true },

  status: {
    type: String,
    enum: ['PENDING_CLEARANCE', 'ACTIVE', 'COMPLETED', 'MAINTENANCE_REQUIRED'],
    default: 'PENDING_CLEARANCE'
  }
}, { timestamps: true });

module.exports = mongoose.model('Lease', leaseSchema);
```

---

4. Advanced Booking Engine Logic (The "Lease" Controller)

Specific steps for the `createLease` controller:

1. **Parse Request:** Get `gearId`, `startCycle`, `endCycle`, and user ID from the JWT token.
2. **Fetch Gear & User:** Verify they exist in the DB.
3. **Clearance Check:** Compare the User's `gravityClearance` against the Gear's `requiredClearance`. (e.g., A `CIVILIAN` cannot rent a `COMMANDER` class Jetpack). Throw a `403 Forbidden` if they fail.
4. **Date Overlap Algorithm:** Query the `Lease` collection to find any overlapping dates for that specific `gearId`.
   - *Logic:* `(newStart < existingEnd && newEnd > existingStart)`
   - If overlap exists, throw a `409 Conflict` (Gear is deployed in another sector).
5. **Transaction:** Deduct `creditsBalance` from the User, create the Lease document, and update the Gear's `isAvailable` status if the lease starts immediately.

---

5. Frontend: The "Levitating" Gear Card (React + Tailwind)

```
import React from 'react';

const FloatingGearCard = ({ gear }) => {
  return (
    <div className="group relative w-80 h-[400px] bg-[#121212] rounded-2xl border border-
gray-800 transition-all duration-500 hover:-translate-y-4 hover:shadow-
[0_0_30px_rgba(0,240,255,0.4)] cursor-pointer overflow-hidden flex flex-col items-center p-
6">

      {/* Background glowing orb effect (reveals on hover) */}
      <div className="absolute inset-0 bg-gradient-to-b from-[#7A04EB]/10 to-transparent
opacity-0 group-hover:opacity-100 transition-opacity duration-500 rounded-2xl"></div>

      {/* Holographic Header Bar */}
      <div className="absolute top-0 left-0 w-full h-1 bg-gradient-to-r from-transparent
via-[#00F0FF] to-transparent opacity-50"></div>

      {/* Gear Image Placeholder */}
      <div className="w-40 h-40 mb-6 mt-4 relative transition-transform duration-700 group-
hover:scale-110 drop-shadow-[0_10px_10px_rgba(0,0,0,0.8)]">
         <img
           src={gear.imageUrl || "/default-hoverboard.png"}
           alt={gear.name}
           className="w-full h-full object-contain filter group-hover:drop-shadow-
[0_0_15px_rgba(0,240,255,0.6)] transition-all duration-300"
         />
      </div>

      {/* Data HUD (Heads Up Display) */}
      <div className="w-full relative z-10 flex flex-col flex-grow">
        <h3 className="text-[#E0E6ED] font-bold text-xl uppercase tracking-wider font-
['Orbitron'] mb-1">
          {gear.name}
        </h3>
        <p className="text-[#00F0FF] text-sm tracking-widest font-mono mb-4">
          CLASS: {gear.requiredClearance}
        </p>

        {/* Specs Grid */}
        <div className="grid grid-cols-2 gap-4 mt-auto border-t border-gray-800 pt-4">
          <div className="flex flex-col">
            <span className="text-gray-500 text-[10px] uppercase font-bold tracking-
widest">Thrust</span>
            <span className="text-white font-mono">{gear.thrustPower} kN</span>
          </div>
          <div className="flex flex-col">
            <span className="text-gray-500 text-[10px] uppercase font-bold tracking-
widest">Battery</span>
            <span className="text-white font-mono">{gear.batteryLife} CYCLES</span>
          </div>
        </div>
```

```
      </div>

      {/* Hover action button */}
      <button className="absolute bottom-[-50px] left-1/2 -translate-x-1/2 w-3/4 py-2 bg-gradient-to-r from-[#00F0FF] to-[#7A04EB] text-white font-bold rounded-full opacity-0 group-hover:opacity-100 group-hover:bottom-6 transition-all duration-300 shadow-[0_0_15px_rgba(122,4,235,0.5)] uppercase tracking-wider text-sm">
        Initiate Lease
      </button>
    </div>
  );
};

export default FloatingGearCard;
```

---

6. Project Roadmap for Solo Development

- **Phase 1: Orbital Station (Backend Setup)**
  - Initialize Node.js/Express project. Connect to MongoDB.
  - Implement the Mongoose schemas above.
  - Set up basic CRUD routes for the Gear (Add new gear, list gear).
- **Phase 2: Access Codes (Authentication)**
  - Implement JWT Authentication.
  - Create User registration/login routes.
  - Build a middleware to read the JWT, find the user, and attach their `gravityClearance` to the request pipeline.
- **Phase 3: The Engine Room (Leasing Logic)**
  - Write the `createLease` controller (checking overlaps and clearance).
  - Write tests (or use Postman) to prove the overlap logic works and rejects invalid dates.
- **Phase 4: HUD Assembly (Frontend Setup)**
  - Initialize Vite + React + Tailwind. Set up the custom configuration for neon colors.
  - Build the `FloatingGearCard` and a Grid layout to map over data.
- **Phase 5: Data Link (Integration)**
  - Connect React to your Express API using Axios or Fetch.
  - Handle JWT storage (e.g., local storage) and send it in Authorization headers.
- **Phase 6: Launch (Deployment)**
  - Host Backend on Render.
  - Host Frontend on Vercel or Netlify.
  - Use MongoDB Atlas for the database.